Execution-Time Validation of Context-Carrying Action Intents: Detailed Results

Luca D'Agati, Giuseppe Tricomi, Fabio Orazio Mirto, and Giovanni Merlino

This document accompanies the paper and the released artifact. It gives the protocols, the per-condition tables, and the detailed results behind every statement of the paper's evaluation. All values are generated from the frozen per-trial data in `data/execution-gap/` by `scripts/build_detailed_results.sh`. Experiment identifiers follow the paper; E1c denotes the valid-intent controls of E1.

1. Execution-gap failure modes

Mode	Condition at release	Representative cause	Policy
Temporal obsolescence	Freshness window or deadline has elapsed	Inference, network, or queue delay	Revoke (may record EXPIRED)
Context drift	A precondition that held at t_d is false at t_v or t_g	Operator override or plant transition	Revoke
Resource conflict	Concurrent intents target the same resource	Multiple AI, vision, or human sources	Order/delay
Semantic fragmentation	Producers encode validity incompatibly	Middleware and application silos	Normalize in AIS
Unsafe delegation	A decision output is treated as authorization	Missing validation boundary	Reject before release

2. Suites and protocols

Suite	Type	Protocol
E0	det.	Fourteen in-process lifecycle and contract regressions (authorization, commit, release, and effect as distinct states).
E1	det.	Intent built at t_d, 200 ms pause, PAUSED written through the gateway, submission; freshness 1000 ms; n=100 per mode.
E3	det.	AI and XR intents on one actuator at equal priority in one batch; declared tie-break.
E4	perf.	10 000 sequential intents to the XAIR core, distinct targets.
E6	fault	Stack in a network namespace, `tc netem` on its loopback; 30 drifted + 10 valid per configuration; windows 500 ms and 10 s.
E8	sim.	E1 pattern against a headless Gazebo Harmonic cell (ROS 2 Jazzy container); ROS message witness and joint-motion witness.
E9	det.	Remote write to the shared snapshot only; delay δ before submission; four policies; n=10 per cell.
E10	timing	50 ms induced gate delay (or none, "natural window"); invalidating write at an offset after validation; classification completed before t_g / overlapping $[t_g, t_m]$ / started after t_m by the gateway clock, and exact ordering by store version; optimistic release and release after an atomic authorization commit.

Continued on next page

Suite	Type	Protocol (continued)
E10-ABA	timing	Line paused 10 ms and resumed 25 ms after submission while the gate waits 50 ms; XAIR read-set, truth, and predicate-only gates (optimistic), read-set and truth gates (atomic), and the policy engine with and without its read-set rule; each flip trial placed by store versions; 50 flips and 25 controls per gate.
E10-deadline	timing	Deadline 100 ms; induced gate delay swept 75–105 ms in 0.5 ms steps; ten trials per step and release path (optimistic, atomic, atomic with release guard, atomic with 3 ms commit margin); age at the check (gateway or store clock) and at t_m (shared realtime clock).
E11	stoch.	Mixed valid/drifted trials, random pause, predicate count, and source; three seeds.
E12	perf.	Closed loop, persistent pool of 1/10/50 producers, 1/64 KiB snapshots, five repetitions.
E13	det.	Five malformed templates, false precondition, duplicates, clock skew.
E14	det.	E1 pattern for four action classes.
E15	det.	Remote write carried by a local OPC UA server.
E16	perf.	100 valid intents per condition under background writers at 0–500 Hz; global vs. read-set version (single host), plus predicate-truth version and policy engine with and without its read-set rule (distributed), and global, read-set, and truth versions on the physical nodes and the embedded boards.
E16-trace	perf.	UCI hydraulic test rig (DOI 10.24432/C5CW21), five 60 s cycles replayed without interruption as OPC UA-style notifications (deadband 0) at 1000/100/10 ms; the 1200 intents of each interval (twelve cells, n=100) are interleaved at random and spread uniformly over the 300 s replay with a random phase; discrete and continuous intents; global, read-set, predicate-truth, and predicate-only scope, and the policy engine with and without its read-set rule.
E17	det.	Producer omits the line-state precondition; drift and valid trials with and without a server-side policy; n=100 per cell.
E18	fault	Actuation log against the store: gateway crash after commit, retried commit, consumer crash between effect and offset update, consumer restarts, concurrent committers, read-set change after commit; 10 repetitions of 100 commits.
E18b	fault	Store with append-only file fsynced on every write; committer and consumer killed with SIGKILL and restarted, store killed and restarted twice per run; SQLite-backed idempotent actuator; five runs of 1000 commits.
E20	trace	Job-shop event log (4TU, 89 days, 31 resources): offline drift prevalence for latencies 1 s–1 h, exact over the timeline (one-minute timestamps), under three decision models (uniform over idle time, idle intervals of at most 8 h, one decision at the start of each idle interval) and three invalidation proxies (the machine leaves idle, the next step is for a different part, the next step is a breakdown), with a Monte Carlo cross-check; live replay of 30 days, START_JOB on idle machines after 30 or 240 plant minutes, five gates: v1.6 rotation design at 8640× (each decision through one gate, 200 per gate and latency) and v1.7 paired design at 4320× (2000 decisions per latency, each through all five gates).
E21	timing	OPC UA cell controller owning the line state (10 ms scan, asynchronous mirror to XAIR); operator pause at the controller 0–30 ms after submission; unconditional command, controller-local interlock (executed only on a running line), and version-conditional command, optimistic and atomic; 20 injected trials per offset and path, 20 controls. E21-ABA: pauses 0–20 ms after submission, each followed by a resume 3 ms later (10 ms on the LAN, 20 ms over the wide-area overlay); an execution on a running line at a controller version newer than the one the check observed is a history violation.

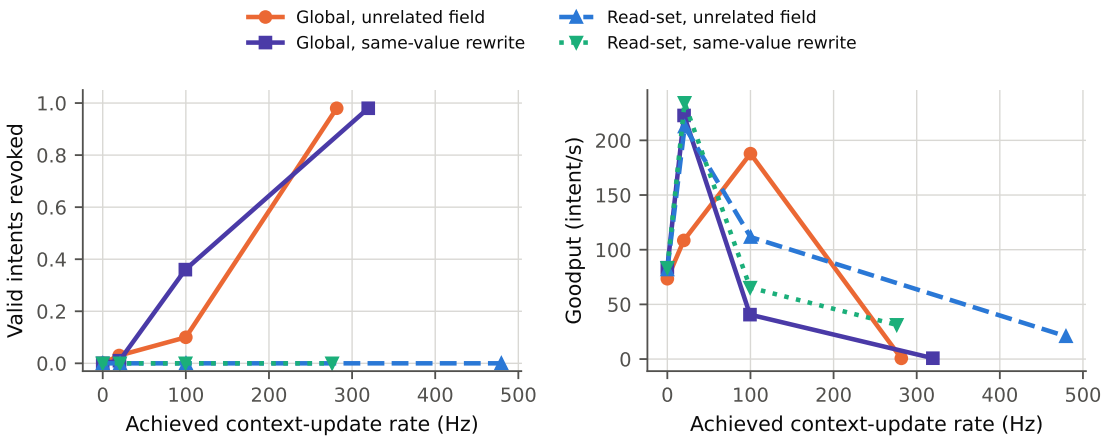
Continued on next page

Suite	Type	Protocol (continued)
E22	physical	Gateway and cell controller on a separate machine (Apple M4). Wide area (TCP connect 62 ms): E1, E1c, E10 natural window, E10-ABA (flip at 280/450 ms, gate delay 400 ms; on both physical nodes each flip trial is placed by store versions and repeated until every gate has 50 trials whose flip falls between validation and the check), E16 (0–20 Hz), E21 (pauses 0–200 ms) and E21-ABA (pauses 80–200 ms). Wi-Fi LAN (TCP connect 6.6 ms median), with XAIR, Redis, and the producers on a Raspberry Pi 4: the same suites, E10-ABA with flip at 80/130 ms and gate delay 200 ms, E16 (0–100 Hz), E21 (pauses 0–120 ms) and E21-ABA (pauses 0–80 ms).
E23	scale	Dedicated store; 0–10 000 registered predicates on as many paths; updates changing 0–100% of those paths (truth preserved); document vs. hash layout; update latency and throughput, validation read, registry size, store memory; retirement with a short retention.
E24	PLC	OpenPLC v3 runtime, Structured Text program in a 10 ms task owning the line state and its version (unconditional, interlock on the line state, or version-conditional; no deadline check in the PLC); Modbus TCP; poller replicating the state to XAIR every 5 ms; E21 protocol with pauses 0–100 ms after submission.
E25	embedded	Raspberry Pi 4 (Cortex-A72, PREEMPT_RT) and VisionFive 2 (SiFive U74, RISC-V); Redis (container on the Pi, built from source on the VisionFive), XAIR, gateway, and harness each pinned to one core, loopback; E4 (three runs of 3000 intents), E11 (three seeds), E10 natural window (both modes), E16 (0–100 Hz; global, read-set, truth).

3. E16 context churn (full)

Target and achieved writer rates, false positives, goodput, and end-to-end p99.

Scope	Pattern	Target [Hz]	Achieved [Hz]	Revoked	Goodput [1/s]	E2E p99 [ms]
Global	same-value rewrite	0	0	0/100	83.5	21.1
Global	unrelated field	0	0	0/100	73.5	25.6
Read-set	same-value rewrite	0	0	0/100	83.4	24.4
Read-set	unrelated field	0	0	0/100	81.9	22.5
Global	same-value rewrite	20	20	1/100	222.8	7.5
Global	unrelated field	20	20	3/100	108.5	23.3
Read-set	same-value rewrite	20	21	0/100	234.3	5.4
Read-set	unrelated field	20	21	0/100	212.6	6.4
Global	same-value rewrite	100	99	36/100	40.6	27.6
Global	unrelated field	100	100	10/100	187.8	6.9
Read-set	same-value rewrite	100	100	0/100	65.3	30.4
Read-set	unrelated field	100	100	0/100	111.6	24.7
Global	same-value rewrite	500	319	98/100	0.8	67.5
Global	unrelated field	500	281	98/100	0.6	57.6
Read-set	same-value rewrite	500	276	0/100	31.3	62.8
Read-set	unrelated field	500	480	0/100	20.8	228.1



Single-host campaign: false revocations (left) and goodput of valid intents (right) against the achieved update rate. Read-set versions revoke none.

4. E6 network impairment

Delay/jitter/loss [ms/ms/%], median end-to-end latency, stale releases, revocation reasons of drifted intents, and valid controls released.

		Window 500 ms			Window 10 s	
Impairment	**E2E p50 [ms]**	**Stale**	**Reasons**	**Valid rel.**	**Reasons**	**Valid rel.**
0/0/0	5	0/30	context 30	10/10	context 30	10/10
10/0/0	85	0/30	context 30	10/10	context 30	10/10
50/10/0	421	0/30	temporal 30	0/10	context 30	10/10
100/10/0	820	0/30	temporal 30	0/10	context 30	10/10
250/50/0	2066	0/30	temporal 30	0/10	context 30	10/10
50/0/0.1	409	0/30	temporal 30	0/10	context 30	10/10
50/0/1.0	408	0/30	temporal 30	0/10	context 30	10/10

5. E8-Gazebo per campaign, E15, E3, and E0

Released, ROS messages observed, ROS-witness agreement, and joint motion per mode. Campaigns whose ROS subscriber stalled are listed but excluded from the witness figures of the paper.

Campaign	**Mode**	**Released**	**ROS msgs**	**Witness agrees**	**Motion**
1	Direct	30/30	30	30/30	29/30
1	Local (coherent)	0/30	0	30/30	0/30
1	Freshness-only	30/30	30	30/30	27/30
1	XAIR	0/30	0	30/30	0/30
2	Direct	30/30	30	30/30	29/30
2	Local (coherent)	0/30	0	30/30	0/30
2	Freshness-only	30/30	30	30/30	28/30
2	XAIR	0/30	0	30/30	0/30

The joint-motion tracker never reports motion without a release and detects 113/120 (94%) releases. Each of the seven misses had zero joint displacement although the command was observed, consistent with an arm already at the commanded pose; the tracker is therefore a conservative proxy, and the message-level witness the reliable one. E14 repeats the E1 pattern for `STOP`, `GRASP`, and `SET_SPEED`

(100/100 vs. 0/100 per condition); E15 carries the remote write over OPC UA (stale and un-notified push caches 30/30, XAIR 0/30); E13 passes 250/250 declared cases; E3 confirms the declared tie-break (100/100); E0 passes ten lifecycle regressions.

6. E14 action classes and E13 fault cases

Scenario	Direct	Local	XAIR
`GRASP`	100/100	0/100	0/100
`RESUME`	100/100	0/100	0/100
`SET_SPEED`	100/100	0/100	0/100
`STOP`	100/100	0/100	0/100

Fault	Passed
clock skew far future	30/30
clock skew future	30/30
clock skew past	30/30
duplicate id	30/30
false precondition	30/30
malformed bad timestamp	23/23
malformed invalid json	24/24
malformed missing fields	22/22
malformed trailing garbage	15/15
malformed wrong types	16/16

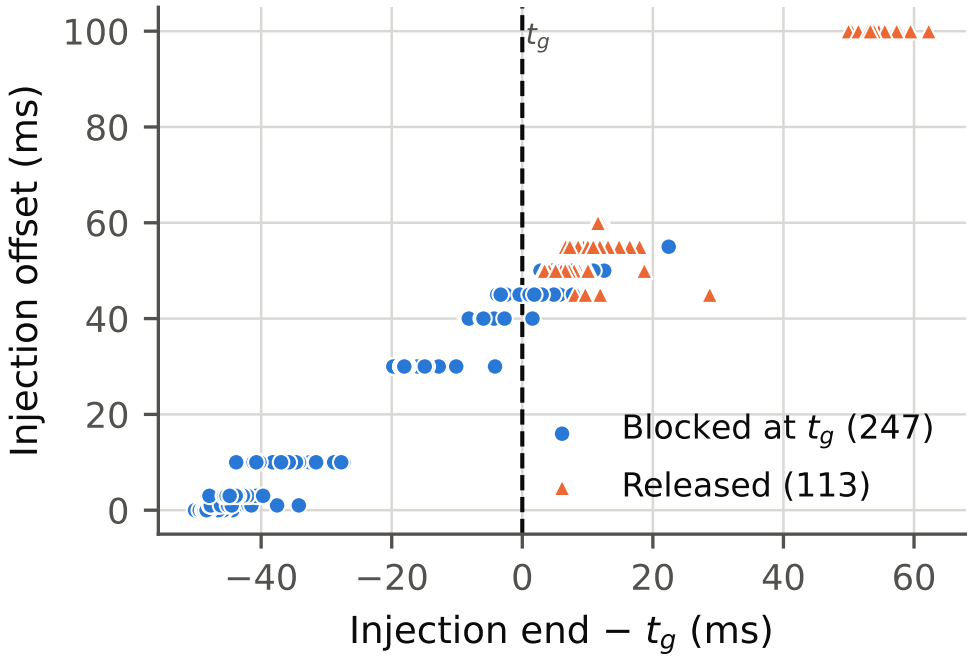
7. E11 validation cost by conjunction length (valid trials)

Predicates	n	p50 [ms]	p95 [ms]	max [ms]
2	38	0.039	0.070	0.070
4	32	0.052	0.088	0.097
8	21	0.072	0.133	0.139
16	21	0.095	0.137	0.156
32	29	0.115	0.156	0.199

8. Single-host E10

Shared host, optimistic release, 50 ms induced gate delay (n=30 per offset); position of each write by the gateway clock (released in parentheses).

Offset [ms]	Before t_g	Overlapping $[t_g, t_m]$	After t_m	Released
0–30	180 (0)	0 (–)	0 (–)	0/180
40	29 (0)	1 (0)	0 (–)	0/30
45	12 (0)	18 (4)	0 (–)	4/30
50	0 (–)	28 (18)	2 (2)	20/30
55	0 (–)	17 (16)	13 (13)	29/30
60, 100	0 (–)	0 (–)	60 (60)	60/60



End of the invalidating write relative to t_g for every injected single-host trial. Gateway-side residual interval, from the end of the recheck to the middleware call: 0.012/0.037/0.200 ms (p50/p99/max over 183 releases); recheck round trip 2.4 ms median.

9. E12 scaling in the three configurations

Median over five repetitions; 500/500 released and no connection retries in every condition. Throughput [intent/s] / p50 / p99 [ms].

		Reserved cores		Distributed testbed	
Producers	**KiB**	**Local auth.**	**XAIR**	**Local auth.**	**XAIR**
1	1	425 / 2.3 / 2.9	295 / 3.3 / 4.1	78 / 12.6 / 15.8	64 / 15.4 / 18.2
1	64	283 / 3.4 / 4.5	225 / 4.2 / 6.9	54 / 18.1 / 21.6	54 / 18.3 / 23.1
10	1	902 / 10.8 / 13.1	554 / 17.4 / 22.1	269 / 36.2 / 48.7	249 / 38.5 / 52.7
10	64	493 / 19.6 / 24.4	361 / 26.6 / 35.1	215 / 44.5 / 59.5	207 / 46.9 / 61.2
50	1	880 / 44.0 / 57.6	506 / 83.8 / 117.0	255 / 167.2 / 241.4	238 / 175.6 / 251.6
50	64	478 / 90.5 / 121.1	342 / 127.9 / 172.7	196 / 218.7 / 340.5	203 / 210.4 / 305.0

Shared host (load average 68–75), for comparison:

Producers	**KiB**	**Local auth.**	**XAIR**
1	1	348 / 2.8 / 3.7	99 / 9.1 / 23.6
1	64	97 / 9.7 / 21.1	85 / 11.2 / 27.5
10	1	299 / 31.8 / 53.0	436 / 22.1 / 29.0
10	64	200 / 46.8 / 69.4	332 / 28.8 / 45.2
50	1	663 / 59.1 / 74.0	193 / 201.4 / 272.0
50	64	419 / 100.2 / 134.4	174 / 218.0 / 326.9

E4 on reserved cores (three runs of 10 000 intents): 909 intents/s, internal p50 0.018 ms, end-to-end p50/p99 0.99/2.86 ms; 956 intents/s, internal p50 0.018 ms, end-to-end p50/p99 0.98/1.41 ms; 922 intents/s, internal p50 0.017 ms, end-to-end p50/p99 1.00/1.48 ms.

10. E16 on the distributed testbed

Scope	Pattern	Target [Hz]	Achieved [Hz]	Revoked	Goodput [1/s]	E2E p99 [ms]
Global	same-value rewrite	0	0	0/100	66.9	19.9
Global	unrelated field	0	0	0/100	64.3	20.6
Policy engine	same-value rewrite	0	0	0/100	107.6	12.0
Policy engine	unrelated field	0	0	0/100	113.0	10.6
Policy engine, read-set	same-value rewrite	0	0	0/100	110.9	11.1
Policy engine, read-set	unrelated field	0	0	0/100	109.7	11.1
Read-set	same-value rewrite	0	0	0/100	65.5	19.3
Read-set	unrelated field	0	0	0/100	66.2	19.5
Predicate-truth	same-value rewrite	0	0	0/100	67.2	17.8
Predicate-truth	unrelated field	0	0	0/100	68.1	16.5
Global	same-value rewrite	20	20	9/100	57.5	21.0
Global	unrelated field	20	20	9/100	59.4	19.4
Policy engine	same-value rewrite	20	20	0/100	105.5	12.2
Policy engine	unrelated field	20	21	0/100	105.2	11.4
Policy engine, read-set	same-value rewrite	20	21	0/100	110.8	11.8
Policy engine, read-set	unrelated field	20	20	0/100	103.1	13.6
Read-set	same-value rewrite	20	20	0/100	64.2	20.1
Read-set	unrelated field	20	20	0/100	65.4	19.9
Predicate-truth	same-value rewrite	20	20	0/100	64.8	21.2
Predicate-truth	unrelated field	20	20	0/100	62.0	22.5
Global	same-value rewrite	100	100	61/100	21.5	23.6
Global	unrelated field	100	100	65/100	18.8	26.4
Policy engine	same-value rewrite	100	100	0/100	103.2	11.6
Policy engine	unrelated field	100	100	0/100	105.6	12.0
Policy engine, read-set	same-value rewrite	100	100	0/100	105.0	11.2
Policy engine, read-set	unrelated field	100	100	0/100	107.6	10.6
Read-set	same-value rewrite	100	100	0/100	53.8	25.2
Read-set	unrelated field	100	97	0/100	51.0	29.2
Predicate-truth	same-value rewrite	100	100	0/100	54.0	25.6
Predicate-truth	unrelated field	100	100	0/100	56.4	21.7
Global	same-value rewrite	500	123	99/100	0.5	24.4
Global	unrelated field	500	122	99/100	0.5	25.3
Policy engine	same-value rewrite	500	128	0/100	104.6	10.6
Policy engine	unrelated field	500	125	0/100	104.0	11.8
Policy engine, read-set	same-value rewrite	500	123	0/100	102.6	11.9
Policy engine, read-set	unrelated field	500	126	0/100	104.7	10.9
Read-set	same-value rewrite	500	124	0/100	49.4	26.6
Read-set	unrelated field	500	123	0/100	49.0	25.7
Predicate-truth	same-value rewrite	500	124	0/100	49.1	24.9
Predicate-truth	unrelated field	500	120	0/100	50.2	24.5

Campaign 1 of the distributed testbed. Distributed E1 and E9 reproduce the single-host counts exactly (direct and freshness-only 100/100, contextual policies 0/100, FPR 0/100; E9 60/60, 20/60, 0/60, 0/60). Across the five campaigns, the global-version rates for the unrelated-field pattern are 9–12% at 20 Hz and 58–68% at 100 Hz, and read-set and predicate-truth versions and the policy engine, with or without its read-set rule, revoke 0/100 in all of their cells. Sensitivity: with 2 ± 1 ms of delay per interface, the natural-window validation-to-release median rises to 13.6 ms (optimistic) and 14.7 ms (atomic), the check-to-t_m bounds to 2.5–6.8 and 2.4–6.8 ms, and the global version revokes 95–99% at 100 Hz; with CPU sets shared by all components, the natural-window figures stay within 0.2 ms of the baseline and the global version revokes 79–82% at 100 Hz; in both variants no release followed a write committed before its check or before t_m. Testbed: containers on disjoint CPU sets, 0.5 ± 0.1 ms of shaped delay per interface, TCP connect 1.2–1.3 ms median; the store-side monotonic timestamps fell inside the requesting interval in 50/50 checks at the start of every campaign.

11. Distributed testbed: per-offset E10, policy, and faults

Campaign 1, per offset (30 injected writes per offset and mode). Clock position of the optimistic trials by the gateway clock alone (completed before t_g / overlapping $[t_g, t_m]$ / started after t_m); "potential": optimistic releases whose write overlapped $[t_g, t_m]$ on that clock; "stale": releases after an atomic commit although the write was ordered before the commit by store version, which a correct commit refuses.

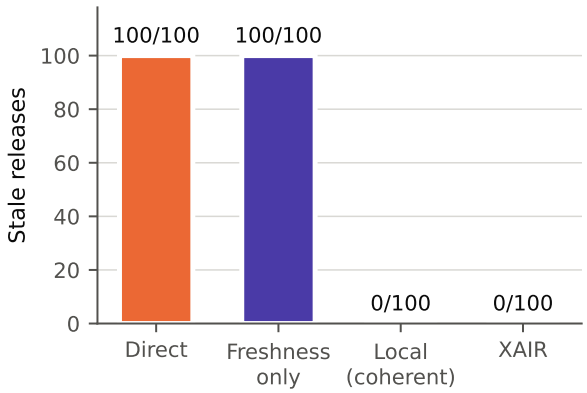
Offset [ms]	Clock position	Opt. released	Potential	Atomic released	Stale
0	30/0/0	0/30	0	0/30	0
30	30/0/0	0/30	0	0/30	0
40	1/29/0	0/30	0	0/30	0
45	0/30/0	1/30	1	0/30	0
50	0/30/0	14/30	14	15/30	0
55	0/9/21	30/30	9	30/30	0
60	0/0/30	30/30	0	30/30	0
100	0/0/30	30/30	0	30/30	0

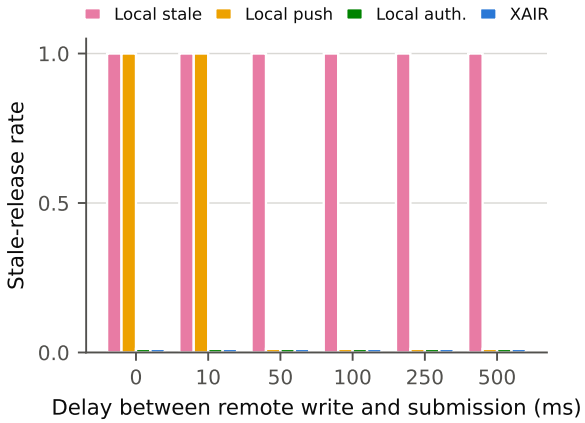
E17 (released / trials): without policy, drift 100/100, valid 100/100; with policy, drift 0/100, valid 100/100.

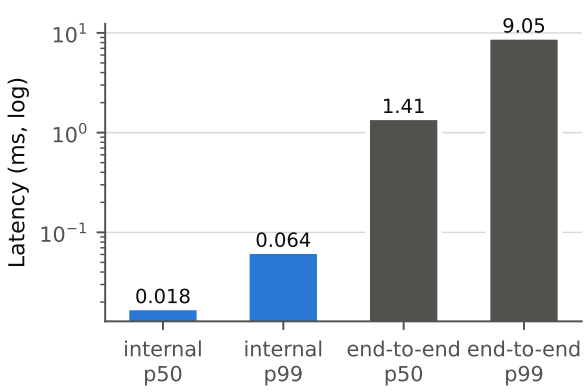
E18 (10 repetitions of 100 commits per scenario; emulated crashes).

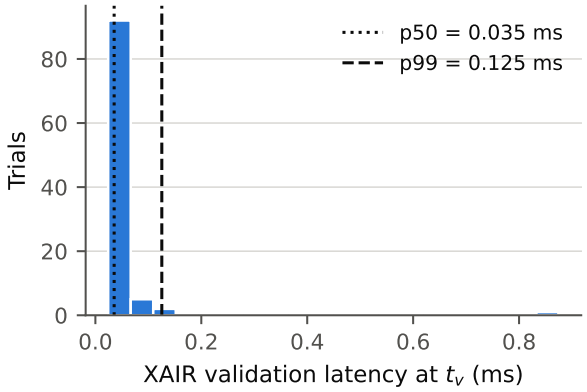
Scenario	Committed	Effects	Duplicates	Lost
8 concurrent committers	1000	1000	0	0
consumer crash after effect	1000	1000	0	0
consumer restarts	1000	1000	0	0
retried commit	1000	1000	0	0
gateway crash after commit	1000	1000	0	0
read-set change after commit	1000	0	0	0

12. Additional figures









Single-host campaign (S), whose latencies are orders of magnitude only: E1 stale releases by mode (100 obsolete intents each); E9 stale-release rate by policy and delay (n=10 per bar; bars at zero are not drawn); E4 internal vs. end-to-end latency (log scale); E1 XAIR validation latency. The paper's performance figures come from the reserved-core and embedded configurations.

13. E1 per testbed

E1: obsolete `RESUME` released / trials, and median end-to-end latency [ms], per testbed (n=100 per mode; 0/100 has 95% upper bound 0.037).

Mode	Single host	Distributed	Physical, wide area	Physical, Wi-Fi LAN
Direct	100/100 / 1.4	100/100 / 3.3	100/100 / 123.1	100/100 / 12.8
Freshness-only	100/100 / 1.5	100/100 / 3.3	100/100 / 123.4	100/100 / 12.7
Local (coherent)	0/100 / 3.7	0/100 / 7.8	0/100 / 247.4	0/100 / 27.5
XAIR	0/100 / 4.3	0/100 / 8.0	0/100 / 249.1	0/100 / 29.6
Policy engine (OPA)	–	0/100 / 6.9	–	–

14. E23 scaling of predicate-truth versions

When an update touches none of the registered paths, it takes 7.0–13 ms in the document layout with 1000 predicates against 1.9–6.0 ms without, and 60–82 ms with 10 000 (ranges over repetitions). In the hash layout it costs no more than without predicates from 1000 predicates on, while an update touching most of them costs more than in the document layout (31–36 against 14–20 ms with 1000). Retirement with a short retention returned 1000 unused predicates to one.

E23: cost of predicate-truth versions (n predicates on n paths; share of paths changed per update, truth preserved; three repetitions). Update median (range over repetitions) and maximum [ms], observations, updates/s, validation read [ms], size [KiB], and update median without predicates (range over repetitions) [ms].

n	Layout	Chg. [%]	Update	Max	Obs.	Upd./s	Valid.	KiB	No preds
10	document	0	0.8–0.9	6.2	900	1169	0.30	1	0.7–0.8
10	document	10	0.8	7.0	900	1176	0.31	1	0.7–0.8
10	document	100	0.8–0.9	9.3	900	620	0.32	1	0.7–0.8
10	hash	0	1.0	7.7	900	621	0.32	1	0.7–0.8
10	hash	10	1.2–1.3	8.0	900	461	0.32	1	0.7–0.8
10	hash	100	1.5–1.8	7.8	900	390	0.33	1	0.7–0.8
100	document	0	1.4–1.6	17.1	900	390	0.66	8	0.9–1.1
100	document	10	1.6–2.7	8.3	900	345	0.72	9	0.9–1.1
100	document	100	2.1–4.6	8.2	900	256	0.71	9	0.9–1.1
100	hash	0	1.0–1.2	26.4	900	524	0.43	8	0.9–1.1
100	hash	10	1.7–3.5	8.2	900	315	0.46	10	0.9–1.1
100	hash	100	4.2–8.0	35.3	900	123	0.48	10	0.9–1.1
1000	document	0	7.0–13.0	48.6	900	124	4.47	78	1.9–6.0
1000	document	10	8.7–9.4	43.1	900	98	5.12	94	1.9–6.0
1000	document	100	13.6–19.6	57.3	300	51	5.15	94	1.9–6.0
1000	hash	0	2.5–2.9	28.8	900	327	0.94	111	1.9–6.0
1000	hash	10	7.0–12.5	26.3	900	116	1.65	126	1.9–6.0
1000	hash	100	30.7–35.9	69.2	300	26	1.60	126	1.9–6.0
10000	document	0	60.0–82.0	202.3	900	12	52.86	805	19.5–27.3
10000	document	10	61.1–105.4	145.2	300	11	58.36	955	19.5–27.3
10000	document	100	88.1–164.7	225.4	300	7	47.76	963	19.5–27.3
10000	hash	0	11.2–18.2	109.2	900	70	9.23	1235	19.5–27.3
10000	hash	10	32.9–65.5	122.2	300	21	14.64	1321	19.5–27.3
10000	hash	100	179.4–375.3	455.6	300	4	13.31	1329	19.5–27.3

14b. E28 per-root context layout

The context was stored per root, in a hash whose entries hold the subtree and the path versions of one root, with a version counter per root for the index of registered predicates; a gate read returns only the roots its predicates read. One writer, fence off, stop fence disabled as in E23; 40 flip trials and 60 controls per gate and cell (two gates), 120 controls per cell. With updates that changed one path (ten at 10 000 predicates; 0.1 % of the paths) every cell of the per-root layout had 80 of 80 flips inside the interval, none released and none of 120 controls revoked; the document and hash layouts at 10 000 predicates had 3–5 and 0 of 80 inside the interval (the lag moved the rest out of it).

E28: update latency by layout, with updates that changed 0.1 % of the paths. Registered predicates, layout, target rate [Hz], achieved rate [updates/s], update median and 99th percentile [ms] (data: `e28_touch01pct.*`).

n	Layout	Target	Achieved	p50	p99
1000	document	20	20.0	9.4	53.5
1000	document	100	66.6	11.6	62.9
10000	document	20	6.5	155.0	242.5
10000	document	100	6.5	155.7	246.6
1000	hash	20	20.0	5.1	21.8
1000	hash	100	100.0	4.6	14.0
10000	hash	20	19.4	43.5	97.1
10000	hash	100	21.0	45.8	95.4
1000	shard	20	20.0	1.9	3.7
1000	shard	100	100.0	1.6	3.1
10000	shard	20	20.0	3.3	5.6
10000	shard	100	100.1	2.7	4.5

With a tenth of the paths changed per update (1000 paths at 10 000 predicates) the per-root layout took the same as the hash layout, since each changed path re-evaluates its predicates (data: `e28_shard_touch10.*`):

n	Layout	Target	Achieved	p50	p99
1000	shard	20	20.0	9.1	19.9
1000	shard	100	98.8	8.2	20.1
10000	shard	20	9.1	114.5	171.5
10000	shard	100	9.8	107.8	160.6

15. E21, E22, and E24 by pause offset

Stale effects [%] of unconditional commands (optimistic release) by the offset of the pause after submission [ms], as offset: rate. A pause before XAIR's check is blocked, a pause after the effect is harmless, and only pauses in between, a window set by the replication and transport delays, act on a paused line; the pooled rates of the paper depend on how many offsets fall in that window.

Testbed	Offset [ms]: stale effects [%]
Distributed (E21)	0: 0; 2: 0; 4: 37; 6: 96; 8: 100; 10: 99; 12: 94; 14: 81; 16: 50; 20: 10; 30: 0
Wide area (E22)	0: 0; 20: 0; 40: 0; 60: 0; 80: 5; 100: 0; 120: 5; 160: 100; 200: 100
Wi-Fi LAN (E22)	0: 0; 10: 0; 20: 55; 30: 80; 40: 55; 60: 20; 80: 5; 100: 0; 120: 5
OpenPLC (E24)	0: 90; 10: 60; 20: 0; 30: 0; 40: 0; 50: 0; 60: 0; 80: 0; 100: 0

16. E10-deadline

Sweeping the gateway's delay across a 100 ms deadline (table below), no release was admitted with an age above the deadline at its check. The check is the gateway's temporal test after its store read returns (optimistic) or the commit t_c (atomic), so the optimistic check-to-t_m interval (0.04 ms) excludes the return trip that Δ_r in E10 includes. At t_m, 193 of 1387 atomic releases (13.9%) were late, because t_m follows the commit by about 1.8 ms; a release guard leaves 4 of 1210 and a 3 ms commit margin none of 1097, against 5 of 1237 for the optimistic gate.
E10-deadline: releases whose age exceeded the 100 ms deadline at the check or at the middleware call, and median check-to-t_m interval (five campaigns). Late counts are out of released intents.

Release path	Trials	Released	Late, check	Late, t_m	Check$\to t_m$ [ms]
optimistic	3050	1237	0	5	0.04
atomic	3050	1387	0	193	1.73–1.78
atomic + release guard	3050	1210	0	4	1.74–1.82
atomic + 3 ms commit margin	3050	1097	0	0	1.75–1.76

17. Detailed results

The paper states each result with its decisive figures. Arrival models (Discussion): with mean gate latencies, and excluding cells where the writers saturate and contend with the gate, the better of the Poisson and periodic forms matches the single-host, distributed, trace, wide-area, and LAN cells within 3.5, 6.2, 6.2, 4.0, and 6.7 percentage points; on the embedded boards the periodic form is within 13 points and the Poisson form underestimates by up to 41. An undetected change in the optimistic interval has probability $\lambda_R \Delta_r$, i.e. $1\text{–}2 \cdot 10^{-3}$ per release on the distributed testbed and $2\text{–}4 \cdot 10^{-3}$ on the embedded boards for one change per second. This section gives the full account, with every count, range, and testbed detail, in the order of the paper.

Testbeds All components run on Python 3.12 with FastAPI and Redis 7.4, on six testbed configurations (S, D, R, P, L, B; B comprises two boards) and an OpenPLC runtime. The *single-host*

configuration (S) runs every suite on loopback of a shared AMD EPYC 7313 host (load average 68–75); its latencies are orders of magnitude only, and the performance figures below come from the other configurations, except the per-predicate cost of E11. The *distributed* configuration (D) places Redis, the policy engine, XAIR, the gateway, an emulated cell controller, and the producers in containers with disjoint CPU sets and 0.5 ± 0.1 ms of shaped delay per interface. It hosts five campaigns and three sensitivity variants (2 ± 1 ms delay, shared CPU sets, phase-locked replay), and its containers share one monotonic clock, verified at the start of every campaign. A *reserved-core* configuration (R) repeats E4 and E12. Two *physical* testbeds move the gateway and the controller to a separate machine (Apple M4), where only store versions order events. On the *wide-area node* (P; TCP connect 62 ms median), XAIR, Redis, and the producers stay on the EPYC host; on the *LAN node* (L; Wi-Fi, TCP connect 6.6–6.8 ms median, 10.7–14.5 ms p99 in its two runs), they run on a Raspberry Pi 4. E22 denotes the suites repeated on P and L. On the *embedded* testbed (B; E25), Redis, XAIR, the gateway, and the harness run on one edge board, each pinned to its own core: a Raspberry Pi 4 (Cortex-A72, PREEMPT_RT kernel) and a VisionFive 2 (SiFive U74, RISC-V). The OpenPLC runtime of E24 and the dedicated store of E23 run in containers on the EPYC host.

Stale actuation and operator policy (RQ1: E1, E14, E8, E17) With n=100 per mode, a 200 ms pause, and a 1000 ms freshness window, both floors release every obsolete `RESUME` and every contextual policy releases none, on the four testbeds and, for the policy engine, on the distributed testbed (Section 13). Valid controls are released 100/100. The pattern holds for three further action classes (E14) and in a simulated Gazebo cell under ROS 2, where an independent subscriber on the actuator topic agrees with the gateway on 240/240 trials (E8). In E17, a `RESUME` that declares only the gripper state is released after a pause in 100/100 trials, since XAIR checks what the producer declared; with an operator policy that makes the line state mandatory, 0/100 are released, and valid controls remain 100/100.

Consistency class (RQ2: E9, E15) In E9 a remote writer updates only the shared snapshot and the intent is submitted 0–500 ms later. The never-refreshed cache releases 60/60 obsolete intents, read-through and XAIR 0/60, and the push policy exactly the 20 trials submitted before its notification, on both testbeds; the delay matters only for the push policy, so E9 establishes the consistency ladder, not a propagation curve. Carrying the write over OPC UA reproduces it (E15).

Optimistic recheck and atomic commit (RQ3: E10) A recheck before release blocked every write ordered before it, but leaves an interval of 0.9–2.3 ms before the middleware call. A thread writes an invalidating update at a controlled offset after validation, while the gateway waits an induced 50 ms before its check or with no induced delay. Each write is ordered exactly against the check, by the global version it created, and located against t_m by shared-clock bounds on its commit (the corresponding table of the paper; 30 writes and 10 controls per offset and campaign). In both modes all 1551 writes ordered before the check were blocked. No injected write landed in the interval between the check and t_m: the writes take a network round trip to commit, and the interval is short. Δ_r lies between 1.0 and 2.2 ms at the median, and $(t_c, t_m]$ between 0.9 and 2.3 ms. E10 therefore measures the width of the interval, not its rate of stale releases; that changes in it do reach the actuator is shown at the controller, where the replica lags by more (E21). On the gateway clock alone, 463 of the 879 optimistic releases could only have been classified as potentially stale; store versions and the shared clock resolve all of them. Without the induced delay, the median validation-to-release time of valid controls was 4.26–4.33 ms per campaign in the optimistic mode and 4.34–4.49 ms with the atomic commit (30 controls per campaign), 0.01–0.19 ms more in the same campaign: one request replaces the gate's snapshot read. On the physical nodes validation to release takes 124–125 ms on the wide-area node and 15 ms on the LAN node, and every write ordered before the check was blocked. Without a shared clock, a write

between the check and t_m cannot be located there, so these nodes test only the ordering against the check. On the embedded boards (E25), which share one clock, the interval is 2.0–3.6 ms (Raspberry Pi 4) and 2.4–4.2 ms (VisionFive 2) at the median; no released write was committed before t_m, and 6 of 174 overlapped it on the VisionFive 2.

Deadline at release (RQ3: E10-deadline) No release was admitted with an age above the deadline at its check. At t_m, 193 of 1387 atomic releases (13.9%) were late, because t_m follows the commit by about 1.8 ms; a release guard or a 3 ms commit margin keeps the deadline at t_m (4 of 1210 and 0 of 1097 late, against 5 of 1237 for the optimistic gate; Section 16).

History violations undone before the check (RQ3: E10-ABA) While the gateway waits, the harness pauses the line and resumes it before the check, so every predicate holds at the check although it was false in the interim (the corresponding table of the paper). These are history violations, not stale releases under the pointwise definition of the paper. Each trial is placed by store versions and tests a history violation only if both writes fall between the validation snapshot and the check; on the distributed testbed all did. Read-set and predicate-truth versions block every such intent in both modes, and so does the policy engine once its rule compares the read-set version it computes from the replicated per-path versions; the predicate-only gate and the policy engine without versions release 250/250. Versioning is therefore a property of the check, not of the runtime that performs it. On the physical nodes, trials are repeated until every gate has 50 trials inside the window; the versioned gates release 0/50 on both nodes and the predicate-only gate 50/50. In 11 wide-area trials and 1 LAN trial, validation came after the pause; they are excluded.

The last check at the controller (RQ3: E21) A check on XAIR's copy lets about half of the unconditional commands act on a paused line; any check in the controller's scan removes these effects. In E21 an emulated OPC UA cell controller owns the line state, executes commands at its 10 ms scan, and replicates its state to XAIR asynchronously. An operator pause is written at the controller 0–30 ms after submission, and ground truth is the controller's state at the effect (the corresponding table of the paper). Three ways of handing the command to the controller follow XAIR's check, in either release mode: an unconditional command, a controller-local interlock that executes only on a running line, and a command conditional on the controller's version as the check observed it. With unconditional commands, 567 and 569 of 1100 injected trials took effect on a paused line (48–54% per campaign), because XAIR's copy lags the controller. The rate is not a property of the system but of where the pauses fall (Fig. 4d of the paper; Section 15): a pause written up to 2 ms after submission reaches XAIR before its check and is blocked, one written 6–12 ms after escapes the check in 94–100% of trials (optimistic release; 92–100% atomic), and at 20 and 30 ms it comes after the effect in 90 and 100% of trials. The unprotected window thus spans about 4–20 ms on this testbed, 20–80 ms on the LAN, and from about 160 ms on over the wide-area overlay, and its position follows the replication and transport delays of each configuration. Both controller-side paths remove stale effects, as their construction implies: the interlock refused 556 and 581 commands and the conditional command 576 and 571, and every valid control executed.

The two differ when the pause is followed, 3 ms later, by a resume (E21-ABA). The interlock then finds a running line and executes: 285 and 297 of 1000 trials took effect after a change that was undone between the check and the effect, a history violation on the controller's state. The version-conditional command refused all of them, as the version had advanced, and produced neither a stale effect nor a history violation. On the physical nodes the pattern repeats for pauses (42 and 37 of 180 stale effects on the wide-area node, 44 and 51 on the LAN node, none for the interlock or the version-conditional command). Pause–resume pairs rarely fit between the check and the effect there, and the interlock let 1–3 of 140 through against none for the conditional command.

The last check on a PLC runtime (RQ3: E24) A PLC runtime reproduces E21: unconditional commands acted on a paused line, while the interlock and the version-conditional command never did. E24 repeats E21 on an OpenPLC v3 runtime executing a Structured Text program in a 10 ms task, reached over Modbus, with a poller replicating its state to XAIR every 5 ms (the corresponding table of the paper). Unconditional commands took effect on a paused line in 30/180 and 29/180 trials, all but one with the pause scheduled at most 10 ms after submission (all written within 30 ms); the interlock and the version-conditional command in none, the PLC refusing 34–38 per path. The atomicity of check and effect comes from the runtime's scan cycle. The program checks only the version: its 16-bit registers carry no absolute time, so in E24 the deadline holds at the gateway's check or the commit, not in the scan (the E21 controller checks both). Its 16-bit version changes at most once per scan, so a value cannot recur within about 327 s at the nominal period, far longer than any trial's span from the replica read to the effect. The runtime runs in a container, and the effect is a register write, not physical output.

Version scope under churn (RQ4: E16, E16-trace) Only a global version turns unrelated updates into revocations, and more so the slower the gate. E16 submits 100 valid intents per condition while background writers update the snapshot at 0–500 Hz, changing an unrelated field or rewriting the line state with its current value, so every revocation is a false positive (the corresponding table of the paper). A global version revokes 12% of valid intents at 20 Hz, 64% at 100 Hz (62–70% per campaign), and 99.5% (99–100% per campaign) once writers saturate near 120 Hz; read-set and predicate-truth versions and the policy engine, with or without its read-set rule, revoke none. The global rate grows with the gate latency on every testbed: 10–36% at 100 Hz on the faster single host, 94–99% on the embedded boards (gate 5–10 ms), all at 100 Hz on the LAN node (about 35 ms), and 96–98% already at 5 Hz on the wide-area node (over 120 ms); read-set and truth versions revoke none on any of them (Section 10).

On a recorded trace, read-set versions still revoke continuous intents whose signal changes value constantly; predicate-truth versions do not. E16-trace replays five 60 s cycles of a public hydraulic test-rig recording (17 sensors at 1–100 Hz) as the notifications of an OPC UA subscription with zero deadband at 1000, 100, or 10 ms. Discrete intents read only the line and gripper state, which the trace never writes; continuous intents also require a 100 Hz pressure to stay below 250 bar, which it always does (the corresponding table of the paper). Read-set versions remove false revocations for discrete intents but not for continuous ones: the pressure changes value in almost every notification, so at 10 ms 37% of valid continuous intents are revoked (34–41% per campaign), as many as under a global version, and 26% with the policy engine's read-set rule, whose gate is faster. Predicate-truth versions remove them, since the threshold is never crossed, as do the gates without versions. With a phase-locked submission the 10 ms rates are similar (37% read-set).

Prevalence in a production log (RQ4: E20) In a production log, drift within a minute is rare unless decisions are taken the moment a machine becomes idle; for decisions that wait tens of minutes it ranges from a few percent to a large share, depending on how the plant schedules (the corresponding table of the paper). E20 uses the event log of a job shop (4543 work steps on 31 resources over 89 days, one-minute timestamps). With decisions spread uniformly over the idle time between recorded steps, the machine leaves the idle state within a decision-to-release latency ℓ for 0.13% of decisions at 1 min, 3.0% at 30 min, and 5.4% at 1 h (0.08, 1.7, and 3.1% if the idle time before each machine's first and after its last step, which the log does not describe, is included). Without idle intervals longer than 8 h, mostly nights and weekends, the fractions rise to 19 and 32% at 30 min and 1 h (under 1% at 1 min); a scheduler that decides as soon as a machine becomes idle sees 35 and 43% (14% at 1 min, a figure bounded by the log's resolution). Narrower proxies give 1.1–9.8% at 30 min to 1 h when the next step is for a different part, and at most 1.1% when it is a breakdown; across resources, the uniform fraction

at 1 h ranges over 0.13–14%. In a live replay of 30 days a scheduler starts jobs on idle machines after 30 or 240 plant minutes. In the first (v1.6) replay, 8640 times faster, decisions went to the five gates in rotation (200 per gate and latency): the direct floor released all 18 of its obsolete decisions and the XAIR gates none of 62; the global version also revoked 5/195 and 6/191 valid decisions, the read-set and truth versions none of 1152. Because each gate saw different decisions and few were obsolete, the replay was repeated with a paired design (v1.7, `data/execution-gap/distributed/e20_paired/`, 4320 times faster): each of 2000 decisions per latency went through all five gates with the same decision time, machine, and precondition, and 35 (30 min) and 147 (240 min) were obsolete for every gate (2 and 1 decisions whose ground truth differed between gates excluded). Direct release released all of them and every XAIR gate none, which bounds each XAIR gate's stale-release rate below 9.9 and 2.5% (Wilson 95%); the global version revoked 41/1963 and 39/1852 valid decisions (2.1% each), the read-set and truth versions none, in the atomic mode as well. The offline and sensitivity files the paired run rewrote are byte-identical to the first ones. With tens to about 150 obsolete decisions, the replay checks the controlled results on production drift; it does not rank the XAIR gates against each other.

Cost on edge processors (RQ5: E25) On both boards, with the XAIR process pinned to one core and Redis, the gateway, and the load generator each on another core of the four-core board, everything on loopback (the corresponding table of the paper), XAIR validates 167–168 (Raspberry Pi 4) and 148–152 (VisionFive 2) sequential intents/s, a sixth to a fifth of the same single-process runtime on reserved EPYC cores below. Each request is a validation (authorization) without release. Internal validation takes 0.10–0.13 ms and the end-to-end path 5.7–6.4 ms at the median. Predicate cost grows from 0.11–0.16 ms for 2 predicates to 0.28–0.68 ms for 32. Validation to release takes 5.0–6.2 ms in either release mode. The slower gate raises the cost of a global version, as E16 showed: 94–99% of valid intents are revoked at 100 Hz, against 64% on the distributed testbed, while read-set and truth versions revoke none. XAIR's resident memory is 93–96 MB, most of it the Python runtime.

Cost, scaling, and faults (RQ5: E4, E11, E12, E18, E6) On reserved cores XAIR validates 909–956 sequential intents/s, with 0.017–0.018 ms internal and about 1.0 ms end-to-end median latency. On the single host, predicate cost grows from 0.039 ms for 2 predicates to 0.115 ms for 32 (E11). In closed loops of 1–50 producers (E12), every condition releases 500/500; on reserved cores read-through has a 1.2–1.9× lower median latency than XAIR, as its one fewer exchange predicts, and on the distributed testbed a similar throughput. The policy-engine path is faster still (99–600 against 54–249 intents/s on the distributed testbed) but does less per intent: two small queries on in-memory data, with no lifecycle, idempotency, or release records, and no store read. Its data are replicated by XAIR's write path, which E12 does not load. The actuation log lost no command and produced no second effect under emulated crashes (E18) and under real ones (E18b: 5000 commits, with the committer, the consumer, and the store killed abruptly 14, 43, and 10 times). This does not show exactly-once physical effects: an actuator whose motion and record are not one atomic step can crash between them. Under network impairment no drifted intent was released, but a 500 ms window shorter than the transport delay revoked 50/70 valid controls (E6); regression suites for malformed inputs, tie-breaks, and the lifecycle pass (E13, E3, E0).

Scaling of predicate-truth versions (RQ5: E23) With a single writer and a thousand registered predicates, one per context path, the default (document) layout applied 78–136 updates/s when each update changed at most a tenth of those paths, and 44–64 updates/s when it changed all of them; the hash layout applied 80–379 and 25–30 updates/s (E23; three repetitions on a loaded host). An update that touches none of them costs 7.0–13 ms in the document layout with 1000 predicates against 1.9–6.0 ms without, and no more than without predicates in the hash layout; layouts, retirement, and timings up to 10 000 predicates are in Section 14.

18. Tables summarized in the paper

The paper states these results in its text and in its key-results figure; the full tables follow.
E10 on the distributed testbed, pooled over five campaigns: injected writes relative to the check (store version) and to t_m (shared clock); interval rows: natural window, ranges of per-campaign medians. Controls: 400/400 and 150/150 released per mode.

Window		**Optimistic**	**Atomic**
induced	blocked / ordered before check	680/680	690/690
	released (write ordered after check)	520	510
	write before / overlapping / after t_m	0 / 0 / 520	0 / 0 / 510
natural	blocked / ordered before check	91/91	90/90
	released (write ordered after check)	359	360
	write before / overlapping / after t_m	0 / 0 / 359	0 / 0 / 360
check-to-t_m, lower bound p50 [ms]		0.96–1.04	0.90–0.98
check-to-t_m, upper bound p50 [ms]		2.12–2.18	2.18–2.28
validation to release p50 [ms]		4.26–4.33	4.34–4.49

E16: valid intents revoked [%] under synthetic context updates, by gate (1000 per condition: two patterns, five campaigns; the highest target rate reached about 120 Hz).

Gate	**20 Hz**	**100 Hz**	**500 Hz target**
Global version	12	64	99.5
Read-set version	0	0	0
Predicate-truth version	0	0	0
Policy engine (OPA)	0	0	0
OPA, read-set version	0	0	0

E16-trace: valid intents revoked [%] while a recorded hydraulic test rig is replayed as context updates, by gate and publishing interval (500 per condition over five campaigns; about 1, 10, and 77 updates/s; disc.: discrete, cont.: continuous intent).

	1000 ms		**100 ms**		**10 ms**	
Gate	disc.	cont.	disc.	cont.	disc.	cont.
Global version	1	0.4	5	4	38	37
Read-set version	0	0.4	0	4	0	37
Predicate-truth version	0	0	0	0	0	0
Predicate-only	0	0	0	0	0	0
OPA, predicates only	0	0	0	0	0	0
OPA, read-set version	0	1	0	3	0	26

E24: release through an OpenPLC runtime (10 ms task, Modbus TCP): effects on a paused line, commands refused by the PLC or blocked by XAIR, executed controls, and median end-to-end latency of the controls.

Release path	**Stale effects**	**Refused**	**Blocked by XAIR**	**Controls**	**E2E [ms]**
optimistic, unconditional	30/180	0	3	20/20	19
atomic, unconditional	29/180	0	5	20/20	18
optimistic, local interlock	0/180	34	3	20/20	19
atomic, local interlock	0/180	34	2	20/20	18
optimistic, version-conditional	0/180	34	4	20/20	20
atomic, version-conditional	0/180	38	3	20/20	18

19. Review round: pause duration, sampled replica, and Eq. (4) on the boards

Pause–resume pairs executed after an undone pause (history violations), by pause duration and by how the controller published its state to XAIR (all changes, or sampled at the interval given); counts over both release modes, and over the campaigns for the physical nodes.

Setting	Pause [ms]	Unconditional	Interlock	Version
distributed, all changes published	3	51/440	44/440	0/440
distributed, all changes published	10	9/440	7/440	0/440
distributed, all changes published	25	0/440	0/440	0/440
distributed, all changes published	50	0/440	0/440	0/440
distributed, all changes published	100	0/440	0/440	0/440
distributed, sampled every 50 ms	3	113/440	111/440	0/440
distributed, sampled every 50 ms	20	2/440	3/440	0/440
distributed, sampled every 200 ms	3	118/440	121/440	0/440
distributed, sampled every 200 ms	20	0/440	4/440	0/440
wide-area node	5	20/720	16/720	0/720
wide-area node	20	1/1160	4/1160	0/1160
wide-area node	50	2/720	0/720	0/720
LAN node	5	58/640	45/640	0/640
LAN node	10	20/840	25/840	0/840
LAN node	50	0/640	0/640	0/640

Unconditional commands that acted on a paused line with a sampled replica (distributed testbed, both release modes), by pause offset [ms]:

Published	0	20	50	100	150	200	300
every change	0/40	10/40	1/40	0/40	0/40	0/40	0/40
50 ms	40/40	12/40	0/40	0/40	0/40	0/40	0/40
200 ms	40/40	17/40	0/40	0/40	0/40	0/40	0/40

Eq. (4) on the boards, global version, 3×400 trials per rate, pooled over the two write patterns: revoked share [%] measured, and predicted from the validation-to-gate time of the harness and from the distance between the two store reads (median [ms], prediction averaged over trials), for a periodic and a Poisson writer.

Board	Writer	Hz	n	Meas.	Median Δ	Model	Median Δ_g	Model
Pi	periodic	20	2400	12.2	5.6	12.1	5.6	12.1
Pi	periodic	50	2400	37.4	7.0	35.1	6.4	35.1
Pi	periodic	100	2400	96.3	9.4	91.7	9.4	92.4
Pi	poisson	20	2400	13.2	5.7	11.6	5.7	11.6
Pi	poisson	50	2400	31.0	5.9	29.5	6.0	29.7
Pi	poisson	100	2400	69.0	9.6	62.4	10.0	63.3
VF2	periodic	20	2400	9.2	6.0	12.6	6.3	13.2
VF2	periodic	50	2400	30.4	7.3	35.6	6.8	36.5
VF2	periodic	100	2400	93.8	9.2	90.2	9.3	92.1
VF2	poisson	20	2400	11.9	6.0	12.1	6.3	12.6
VF2	poisson	50	2400	28.5	6.1	30.1	6.5	31.2
VF2	poisson	100	2400	65.5	9.2	61.2	9.4	62.6

Stop fence with a skewed producer clock: `RESUME` released after a stop (stale), valid controls released, and valid `RESUME` decided Δ ms after a stop that were released, by the clock bound ε [ms] and the producer offset θ [ms].

ε	θ	Stale	Controls	0	5	20	60	120
0	+55	58/480	31/60	30/60	30/60	30/60	30/60	30/60
60	-55	0/480	59/60	0/60	0/60	0/60	0/60	60/60
60	+55	0/480	60/60	60/60	60/60	60/60	60/60	60/60

Sequential throughput [intents/s] of the embedded-board experiment: first campaign, and the current build without and with the stop fence.

Board	First campaign	Fence off	Fence on	Fall [%]
Raspberry Pi 4	167–168	131–132	106	19
VisionFive 2	148–152	130–131	84–85	35

Headline quantity of each timing experiment over campaigns (original and repeats): store-check experiment, injected writes ordered before the check that were blocked; controller experiment, offset [ms] at which the unprotected interval of the unconditional command opens, last such offset, and last offset with any stale effect; update-rate experiment, valid intents revoked [%] by a global and by a read-set version under 20 Hz of unrelated writes.

Testbed	n	blocked	n	opens	last $\geq$ half	last any	n	global	read-set
P	3	99/99	3	120–160	200	200–280	3	100	0
L	3	102/102	3	20	40	40–120	3	29–74	0
B, Pi	3	104/104	–	–	–	–	3	13–17	0
B, VF2	3	32/32	–	–	–	–	3	0–26	0

Deadline experiment on the physical nodes: commands released of those submitted, and how many were older than the deadline d at the last check and at the middleware call, for the declared ε [ms] and an emulated offset θ [ms] of the producer's clock ahead of true time; last column, largest excess over d at the check [ms].

Node	d	ε	θ	Variant	Released	Late check	Late t_m	Max
wide-area	1000	0	+0	optimistic	27/102	0	0	0
wide-area	1000	0	+0	atomic	9/102	0	0	0
wide-area	1000	0	+54	optimistic	50/102	0	0	0
wide-area	1000	0	+54	atomic	36/102	26	0	53
wide-area	1000	60	+54	optimistic	18/102	0	0	0
wide-area	1000	60	+54	atomic	4/102	0	0	0
wide-area	1000	0	+108	optimistic	76/102	27	27	52
wide-area	1000	0	+108	atomic	60/102	52	8	105
wide-area	1000	60	+108	optimistic	48/102	0	0	0
wide-area	1000	60	+108	atomic	31/102	22	0	47
wide-area	1000	120	+108	optimistic	25/102	0	0	0
wide-area	1000	120	+108	atomic	6/102	0	0	0
LAN	100	0	+0	optimistic	311/420	21	21	4
LAN	100	0	+0	atomic	0/420	0	0	0
LAN	100	40	+0	optimistic	14/210	0	0	0
LAN	100	40	+0	atomic	0/210	0	0	0
LAN	300	0	+0	optimistic	128/128	0	0	0
LAN	300	0	+0	atomic	84/128	0	0	0
LAN	300	40	+0	optimistic	121/128	0	0	0
LAN	300	40	+0	atomic	53/128	0	0	0